

Problem 2 (10 points)

Throughout, the assignment's binary unit convention applies, so $64 \text{ KB} = 2^{16} \text{ Bytes}$ and $2 \text{ GB} = 2^{31} \text{ Bytes}$.

Question A (2 points)

The page offset must address every byte within a page. A page holds $64 \text{ KB} = 2^{16} \text{ Bytes}$, so the offset is 16 bits, and the remaining $30 - 16 = 14$ bits of the 30-bit virtual address form the virtual page number.

Table 1: virtual address components

Page offset / bits	Virtual page memory (VPN) / bits	Total / bits
16	14	30

Question B (4 points)

A single-level page table holds one entry per virtual page, so it is indexed by the 14-bit VPN from Question A; the physical memory size does not determine the number of entries.

Number of PTEs: $2^{14} = 16,384$ entries.

Physical memory for the page table: $2^{14} \times 4 \text{ Bytes} = 2^{16} \text{ Bytes} = 64 \text{ KB}$.

Question C (4 points)

Pros.

1. *Smaller page table.* A larger page size widens the offset field and narrows the VPN by the same number of bits, so the entry count falls geometrically. Doubling the page size in Question B to 128 KB would leave a 13-bit VPN, halving the table from 64 KB to 32 KB.
2. *Greater TLB reach.* Each TLB entry maps a whole page, so with larger pages a fixed number of TLB entries covers proportionally more of the address space. That lowers the TLB miss rate for programs with large working sets, and each avoided miss avoids a page-table walk.

Cons.

1. *More internal fragmentation.* Memory is allocated in whole pages, so the unused remainder of a partially filled page is wasted. The waste per incompletely filled region grows with the page size, which hurts most for many small objects or sparse address spaces.
2. *Costlier page faults.* A fault transfers an entire page from disk, so a larger page means a longer transfer and more memory traffic. If the program only touches a small part of that page, the extra bytes fetched are wasted I/O and the fault latency rises.